

PegFactory-as-a-Service & EasyBasket Composer — Cypherpunk2048 Standard, Production Specification

(c) 2026 BANKON — all rights reserved. Apache 2.0. For Gregory "codephreak" — DELTAVERSE / PYTHAI / BANKON / AgenticPlace / mindX ecosystem.

TL;DR

- The right architecture for the PegFactory is a hybrid of Reserve Protocol's permissionless DTF deployer pattern (a Deployer contract that stamps out fully-collateralized basket tokens governed by collateral plugins implementing `refresh / status / price / refPerTok / targetPerRef`) and Set Protocol V2's modular SetToken / Module / Controller separation, fused with Hyperliquid HIP-3 oracle clamps (± 50 bps per relayer update, $\pm 1\%$ per mark step, ≥ 2.5 s minimum gap, $\pm 10\times$ start-of-day rail) so pre-IPO and 24/7 RWA pegs remain manipulation-resistant. The factory itself uses OpenZeppelin's `Clones` library (EIP-1167) with CREATE2 salts for deterministic addresses, identical in spirit to Uniswap V2's pair stamping but with an explicit `initialize(BasketParams)` initializer.
- EasyBasket is expressed on-chain as an ordered array of `BasketComponent { bytes32 assetKey; uint192 targetUnitsPerBasket; address oracleAdapter; address collateralPlugin; uint16 weightBps; }`. NAV uses Reserve's closed form (sum of `refPerTok * targetPerRef * unitsPerBasket`) with a parameterized unit-of-account per peg (USD, BTC, XAU, EUR), promoting the BitcoinAnchorOracle pattern from a one-off to a first-class basket primitive.
- Ship in three releases. T+30: deploy `peg_factory`, `peg_token_template`, `oracle_aggregator`, four metal plugins (PAXG-XAU, KAG-XAG, Pyth XPT, Pyth XPD), and the `ebPM` reference peg on Ethereum and Base. T+60: cash-mint via Uniswap V4 hooks, KYC TransferGate wired to BONAFIDE Fides, `ebSPCX+` and `ebFX5`. T+90: Algopy parallel on Algorand mainnet, x402 metering via GoPlausible, AgenticPlace agent-registry hooks, mindX MCP tool generation, and the DELTAVERSE Debt Inheritance plugin family. Chainlink Proof of Reserve Secure Mint is wired into every physically-backed mint path from day one to eliminate infinite-mint attacks.

Key Findings

Live reference designs and where each one wins

Reserve Protocol is the single closest production analogue. Its docs state: *"In a similar way as how anyone can create a new trading pair on Uniswap, anyone can permissionlessly create a new Reserve stablecoin (RToken) by interacting with Reserve Protocol's smart contracts. The protocol applies a system of factory smart contracts that allows anyone to deploy their own smart contract instance."* The key portable innovation is the **Collateral Plugin** abstraction: every backing asset is

wrapped in a small contract implementing `refresh()`, `status()` returns (`SOUND | IFFY | DISABLED`), `price()` returns (low, high) bracket, `targetName()`, `refPerTok()`, and `targetPerRef()`. Reserve makes the `refresh()` contract explicit: *"In any valid implementation, status() MUST become DISABLED in refresh() if refPerTok() has ever decreased since last call."* This is exactly what is needed for tokenized metals and pre-IPO equity — the basket cannot silently revalue downward, and the protocol can programmatically trip into recapitalization. Reserve's Index DTF FolioDeployer is live on Base at `0x3451fD177E9a8bB4Eb8271E627A804BD22A816F9` (v5.0.0, per docs.reserve.org/core-components/index-dtfs/smart-contracts.md) and on Ethereum at `0x4d201a6e5bf975e2cee9e5cbdfc803c0ff122073` (v5.0.0). (Reserve + 5)

Set Protocol V2 / Index Coop is the second closest reference and contributes the **modular module pattern**: a SetToken is an ERC-20 that owns its own component balances and delegates privileged operations (mint, burn, invoke, reconfigure) to whitelisted modules registered against a global Controller. DPI at `0x1494ca1f11d487c2bbe4543e90080aeba4ba3c2b` and MVI at `0x72e364f2abdc788b7e918bc238b21f109cd634d7` both use `DebtIssuanceModuleV2` at `0xd8EF3cACe8b4907117a45B0b125c68560532F94D`. Per the Index Coop governance forum post at gov.indexcoop.com/t/minor-updates-to-the-dpi-and-mvi-methodologies/4814, *"DPI currently has 10 components and MVI 8. The Index Coop Product team has made the decision with immediate affect to cap the number of components in each product to 10"* — an empirically validated upper bound for gas. The lesson: separate token (state) from module (logic) so fee, rebalance, and KYC modules can be hot-swapped without redeploying the ERC-20. (Tokensets + 2)

GMX GLP demonstrates the AMM-style mint-and-redeem-from-pooled-collateral pattern where the LP token is 1:1 collateralized and target weights are governance-defined with rebalancing incentivized through dynamic mint/redeem fees rather than auctions. GMX requires all assets in GLP to have *"Chainlink price feeds with high trading volume and liquidity across several exchanges"*, and per the Exponential DeFi analysis at exponential.fi/assets/gmx-lp-arbitrum, *"GLP rebalances passively once per week by updating asset prices to calculate new target weights."* This is the right model for highly liquid metal baskets where in-kind delivery is impractical. TokenSets, dHEDGE, Enzyme, Balancer weighted pools, and mStable were reviewed; their patterns are subsumed by Reserve + Set V2 + GMX. (Exponential DeFi)

Oracle composition

The Hyperliquid HIP-3 spec, lifted from the trade[XYZ] equity perps deployment, gives the gold-standard recipe for clamping a relayer-fed oracle so it cannot be manipulated even when the underlying market is closed. Per hyperliquid.gitbook.io/hyperliquid-docs/trading/robust-price-indices, *"Oracle prices are updated by the validators approximately once every three seconds"*; the protocol computes a local "best bid / best ask / last trade" median and takes the median of three components (oracle, EMA-of-mid-minus-oracle, local median) as the operational mark. Three rules are enforced atomically: *"Relayer updates are clamped to ±50 bps of the current value"*

to mitigate significant price jumps", "markPx moves are clamped to 1% from previous markPx", and "SetOracle can be called multiple times but there must be at least 2.5 seconds between calls", with a global rail that "all prices will be clamped to within 10 times the start-of-day value". Porting these into a standalone `oracle_aggregator` is the single highest-leverage anti-manipulation primitive in the entire system; it composes naturally with both Chainlink AggregatorV3 (push) and Pyth `getPriceNoOlderThan` (pull). The Pyth EVM contract on Ethereum mainnet is `0x4305FB66699C3B2702D4d05CF36551390A4c69C6`. [Hyperliquid Docs + 4](#)

For the proof-of-reserve circuit-breaker layer, Chainlink Proof of Reserve Secure Mint is production-grade. The CACHE Gold case study confirms: "By integrating Chainlink Proof of Reserve, CACHE Gold can provide its users with stronger assurances that no unbacked CGT can be minted... When the off-chain gold supply would be less than the outstanding supply of CGT after minting, the smart contract halts minting." Wired in via one `require(reserves >= newSupply, "POR_UNDERCOLLAT")` in the mint path against a Chainlink PoR `AggregatorV3Interface`. Chainlink XAU/USD on Ethereum mainnet lives at `0x214eD9Da11D2fBe465a6fC601a91E62EbEc1a0D6` (per chain.link/feeds/ethereum/mainnet/xau-usd), XAG/USD at `0x379589227b15F1a12195D3f2d90bBc9F31f95235`. PAXG at `0x45804880De22913dAFE09f4980848ECE6EcBAf78` has a first-party Paxos PoR feed via Chainlink that we consume verbatim; XAUT at `0x68749665ff8d2d112fa859aa293f07a622782f38` does not have first-party PoR and must be treated as "trust-the-issuer" with a higher safety margin.

[Chainlink + 4](#)

The factory mechanic

EIP-1167 minimal proxies via OpenZeppelin's `Clones.cloneDeterministic(address implementation, bytes32 salt)` are the deployment primitive. A clone's runtime is 45 bytes of `DELEGATECALL` stub forwarding to a single shared implementation, making each peg deployment roughly an order of magnitude cheaper than re-deploying full bytecode. Clones are immutable: their implementation pointer is hard-coded into bytecode. For the per-peg template this is desirable (holders want a frozen, audited contract). The factory itself sits behind a UUPS upgradeable proxy so the template pointer can advance with BANKON releases. [Medium](#)

[Medium](#)

Details

Architectural overview — five layers

At the base sits the **Asset Plugin Layer**, a registry of `ICollateralPlugin` contracts — one per asset class (XAU via Chainlink XAU/USD + PAXG; XAG via Chainlink XAG/USD + KAG; XPT/XPD via Pyth Metal feeds; BTC via Chainlink BTC/USD + optional WBTC; ETH; SOL; fiat baskets; pre-IPO equity references including the SPCX three-layer adapter from prior work; the BitcoinAnchorOracle as a first-class plugin). Each plugin exposes the Reserve-style five-function interface plus an additional `proofOfReserveOK()` boolean returning false when a registered PoR

feed reports under-collateralization. The plugin layer is permissionless to read but permissioned to register: only the BONAFIDE Senatus contract may add a plugin to the canonical registry, preventing a malicious deployer from minting a peg backed by a fake "gold" contract.

Above sits the **Oracle Composition Layer**, where `oracle_aggregator` combines up to four primary sources (Chainlink, Pyth, RedStone, Chronicle) by sorted median with weights, applies HIP-3 clamps (± 50 bps per update, $\pm 1\%$ per mark step, $\pm 10\times$ start-of-day, ≥ 2.5 s gap), enforces heartbeat staleness thresholds (default 60 s for crypto, 300 s for metals during market hours, 24 h on weekends with a "market closed" flag), and exposes a single `latestPriceE18(bytes32 assetKey)` returns `(uint256 price, uint64 publishedAt, uint8 status)`. The circuit-breaker logic lives here: if any component oracle deviates more than the governance-configured threshold (default 2%) from the median of the remainder, the aggregator returns `status = IFFY` and dependent pegs are automatically blocked from minting until refresh.

The **PegTokenTemplate Layer** is the immutable ERC-20 implementation every cloned peg points to. It is initialized once per clone via `initialize(BasketParams calldata)` from the factory in the same transaction as deployment, preventing the well-known uninitialized-clone takeover. It implements ERC-20 plus ERC-2612 permit, plus a transfer hook consulting a `transfer_gate` for KYC / Reg-S / Reg-D / sanctions / per-jurisdiction restrictions. Mint and burn route through pluggable `IIssuanceModule` modules registered per peg, defaulting to `basic_issuance_module` (in-kind component deposit) and `cash_issuance_module` (USDC in, internally swap via 0x or Uniswap V4 to acquire components). Streaming, performance, and exit fees are handled by `fee_router`, splitting accruals across issuer, the protocol treasury (Aerarium / Curia), oracle operators, and an optional RSR-style backstop staking pool.

The **PegFactory Layer** is a single contract per chain exposing `createPeg(BasketParams, bytes32 salt)` external returns `(address peg)`. It enforces the x402 deployment fee (default \$100 USDC, configurable by Senatus), validates that every basket component points to a registered plugin in the BONAFIDE-approved asset library, deterministically computes the CREATE2 salt as `keccak256(abi.encode(msg.sender, symbol, basketHash, userSalt))`, calls `Clones.cloneDeterministic(template, salt)`, calls `initialize` on the clone with the validated parameters, and writes a `PegRecord` into the `peg_registry` containing deployer, basket hash, issuance module, asset class (METAL_BASKET / CRYPTO_BASKET / RWA_EQUITY / SOVEREIGN_FX / MIXED), deployment timestamp, and a Pinata/IPFS CID pointing to the human-readable mandate. The registry is what AgenticPlace queries to discover deployable peg-as-a-service agents.

The **Integration Layer** binds everything to the existing BANKON / DELTAVERSE stack. BONAFIDE Fides (KYC) is consulted by `transfer_gate` before any non-whitelisted transfer. BANKON identity provides the off-chain attestation mapping a wallet to Reg-D accredited status via EIP-712 signed messages verified by Fides. The Genius contract grants Senatus authority to add asset plugins; Tabularium is the canonical event log indexer; SponsioPactum holds the revenue waterfall configuration; Censura is the slashing module used if a peg deployer is found

operating a manipulated oracle; Aerarium / Curia receives the protocol-fee share. The DELTAVERSE Debt Inheritance Protocol's 12 EVM contracts (BitcoinAnchorOracle, DebasementIndexV2, InverseDebtToken, GlobalCurrencyBasket, SovereignDebtRegistry, etc.) are wrapped in `ICollateralPlugin` adapters and registered, so a deployer can compose a peg whose basket includes "10% of GlobalCurrencyBasket as a hedge against any single sovereign's debasement" — compositional power Reserve does not have because its plugins are confined to USD-target ERC-20s.

Solidity source — production-grade

Every file carries:

```
solidity
```

```
// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON — all rights reserved.
// Cypherpunk2048 standard. Solidity 0.8.25+. Foundry-ready.
pragma solidity ^0.8.25;
```

`peg_factory.sol` — entry-point factory:

```
solidity
```

```

import {Clones} from "@openzeppelin/contracts/proxy/Clones.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {peg_registry} from "./peg_registry.sol";
import {asset_library} from "./asset_library.sol";
import {peg_token_template, BasketParams, BasketComponent} from "./peg_token_template.sol";
import {fee_router} from "./fee_router.sol";

contract peg_factory is ReentrancyGuard {
    using SafeERC20 for IERC20;
    address public immutable template;
    peg_registry public immutable registry;
    asset_library public immutable library;
    fee_router public immutable feeRouter;
    address public immutable usdc;
    uint256 public deploymentFeeUsdc6;
    address public senatus;

    event PegCreated(address indexed peg, address indexed deployer, bytes32 indexed basketHash,
        string name, string symbol, uint8 assetClass, string manifestCid);

    modifier onlySenatus() { require(msg.sender == senatus, "ONLY_SENATUS"); _; }
    function setDeploymentFee(uint256 _fee) external onlySenatus { deploymentFeeUsdc6 = _fee; }

    function createPeg(BasketParams calldata params, bytes32 userSalt)
        external nonReentrant returns (address peg)
    {
        if (deploymentFeeUsdc6 > 0) {
            IERC20(usdc).safeTransferFrom(msg.sender, address(feeRouter), deploymentFeeUsdc6);
            feeRouter.routeDeploymentFee(deploymentFeeUsdc6);
        }
        require(params.components.length > 0 && params.components.length <= 10, "BAD_PARAMS_LENGTH");
        uint256 totalWeight;
        for (uint256 i = 0; i < params.components.length; ++i) {
            BasketComponent calldata c = params.components[i];
            require(library.isPluginRegistered(c.collateralPlugin), "PLUGIN_UNREG");
            require(library.assetKeyOf(c.collateralPlugin) == c.assetKey, "PLUGIN_MISMATCH");
            require(c.weightBps > 0 && c.weightBps <= 10_000, "BAD_WEIGHT");
            totalWeight += c.weightBps;
        }
        require(totalWeight == 10_000, "WEIGHTS_NOT_100PCT");
        bytes32 basketHash = keccak256(abi.encode(params.components, params.unitOfAsset));
    }
}

```

```

bytes32 salt = keccak256(abi.encode(msg.sender, params.symbol, basketHash, u
peg = Clones.cloneDeterministic(template, salt);
peg_token_template(peg).initialize(params, msg.sender, address(feeRouter));
registry.recordPeg(peg, msg.sender, basketHash, params.name, params.symbol,
                    params.assetClass, params.manifestCid);
emit PegCreated(peg, msg.sender, basketHash, params.name, params.symbol,
                params.assetClass, params.manifestCid);
    }
}

```

peg_token_template.sol — immutable per-peg ERC-20 logic, cloned by the factory:

solidity

```

struct BasketComponent {
    bytes32 assetKey; // keccak256("XAU"), keccak256("BTC"), etc.
    uint192 targetUnitsPerBasket; // 1e18 = 1.0 unit (oz, share, satoshi) per basket
    address oracleAdapter;
    address collateralPlugin;
    uint16 weightBps;
}

struct BasketParams {
    string name; string symbol; uint8 decimals;
    BasketComponent[] components;
    bytes32 unitOfAccount; // keccak256("USD"|"BTC"|"XAU"|"EUR")
    uint8 assetClass;
    address transferGate;
    address issuanceModule;
    uint16 entryFeeBps; uint16 exitFeeBps;
    uint16 streamingFeeBpsPerYear; uint16 perfFeeBps;
    string manifestCid;
}

contract peg_token_template is Initializable, ERC20Upgradeable, ERC20PermitUpgradeable {
    BasketParams public params;
    address public issuer; address public feeRouter;
    basket_composer public composer;
    uint256 public lastFeeAccrual;
    error TransferBlocked();

    function initialize(BasketParams calldata p, address _issuer, address _feeRouter) public {
        __ERC20_init(p.name, p.symbol); __ERC20Permit_init(p.name);
        params = p; issuer = _issuer; feeRouter = _feeRouter;
        composer = basket_composer(p.issuanceModule);
        lastFeeAccrual = block.timestamp;
    }

    function decimals() public view override returns (uint8) { return params.decimals; }
    function navE18() public view returns (uint256) { return composer.computeNavE18(params); }

    function mint(address to, uint256 amount, bytes calldata oracleUpdateData)
        external payable returns (uint256 paid)
    {
        paid = composer.issue{value: msg.value}(params, msg.sender, to, amount, oracleUpdateData);
        _mint(to, amount);
    }

    function redeem(uint256 amount, address to) external returns (uint256 returned)
    {
        _burn(msg.sender, amount);
        return composer.burn(to, amount);
    }
}

```



```
        returned = composer.redeem(params, msg.sender, to, amount);
    }
    function _update(address from, address to, uint256 value) internal override {
        if (params.transferGate != address(0) && from != address(0) && to != address(0)) {
            if (!transfer_gate(params.transferGate).canTransfer(from, to, value)) re
        }
        super._update(from, to, value);
    }
}
```

basket_composer.sol — NAV / mint / redeem engine modelled on Reserve's collateral and Set's BasicIssuanceModule:

solidity

```

interface ICollateralPlugin {
    enum Status { SOUND, IFFY, DISABLED }
    function refresh() external;
    function status() external view returns (Status);
    function priceE18() external view returns (uint256 low, uint256 high);
    function refPerTokE18() external view returns (uint256);
    function targetPerRefE18() external view returns (uint256);
    function proofOfReserveOK(uint256 supplyAfter) external view returns (bool);
    function erc20() external view returns (address);
    function targetName() external view returns (bytes32);
}

contract basket_composer {
    uint256 constant E18 = 1e18;

    function computeNavE18(BasketParams memory p) public view returns (uint256 navE18) {
        for (uint256 i = 0; i < p.components.length; ++i) {
            BasketComponent memory c = p.components[i];
            ICollateralPlugin plugin = ICollateralPlugin(c.collateralPlugin);
            require(plugin.status() != ICollateralPlugin.Status.DISABLED, "DISABLED");
            (uint256 low, uint256 high) = plugin.priceE18();
            uint256 mid = (low + high) / 2;
            navE18 += ((uint256(c.targetUnitsPerBasket) * mid) / E18) * c.weightBps;
        }
    }

    function issue(BasketParams memory p, address payer, address to, uint256 amount,
        external payable returns (uint256 paid)
    {
        for (uint256 i = 0; i < p.components.length; ++i) {
            BasketComponent memory c = p.components[i];
            ICollateralPlugin plugin = ICollateralPlugin(c.collateralPlugin);
            plugin.refresh();
            require(plugin.status() == ICollateralPlugin.Status.SOUND, "NOT_SOUND");
            require(plugin.proofOfReserveOK(amount), "POR_UNDERCOLLAT");
            uint256 needed = (uint256(c.targetUnitsPerBasket) * amount) / E18;
            IERC20(plugin.erc20()).safeTransferFrom(payer, address(this), needed);
            paid += needed;
        }
        emit Issued(to, amount, paid);
    }

    function redeem(BasketParams memory p, address from, address to, uint256 amount)

```

```

        external returns (uint256 returned)
    {
        for (uint256 i = 0; i < p.components.length; ++i) {
            BasketComponent memory c = p.components[i];
            ICollateralPlugin plugin = ICollateralPlugin(c.collateralPlugin);
            uint256 out = (uint256(c.targetUnitsPerBasket) * amount) / E18;
            IERC20(plugin.erc20()).safeTransfer(to, out);
            returned += out;
        }
        emit Redeemed(from, amount, returned);
    }
    event Issued(address indexed to, uint256 amount, uint256 paid);
    event Redeemed(address indexed from, uint256 amount, uint256 returned);
}

```

oracle_aggregator.sol — the HIP-3-style clamped multi-source aggregator (excerpted):

solidity

```

import {AggregatorV3Interface} from "@chainlink/contracts/src/v0.8/shared/interfaces
import {IPyth, PythStructs} from "@pythnetwork/pyth-sdk-solidity/IPyth.sol";

contract oracle_aggregator {
    struct Source { uint8 kind; address feed; bytes32 pythId; uint32 heartbeatSec; u
    struct AssetConfig {
        Source[] sources;
        uint256 lastPriceE18; uint64 lastUpdate; uint256 sodPriceE18;
        uint16 clampPerUpdateBps; // 50
        uint16 clampPerMarkBps; // 100
        uint32 minGapSec; // 3
        uint16 crossDevBps; // 200
        uint8 status;
    }
    mapping(bytes32 => AssetConfig) internal cfg;

    function updateAndGet(bytes32 assetKey, bytes calldata pythUpdate)
        external payable returns (uint256 priceE18)
    {
        AssetConfig storage a = cfg[assetKey];
        require(block.timestamp >= a.lastUpdate + a.minGapSec, "GAP");
        uint256[] memory raw = new uint256[](a.sources.length);
        for (uint256 i = 0; i < a.sources.length; ++i) {
            Source storage s = a.sources[i];
            if (s.kind == 1) {
                (, int256 ans,, uint256 updatedAt,) = AggregatorV3Interface(s.feed).
                require(block.timestamp - updatedAt <= s.heartbeatSec, "CL_STALE");
                require(ans > 0, "CL_NEG");
                uint8 d = AggregatorV3Interface(s.feed).decimals();
                raw[i] = uint256(ans) * 10**(18 - d);
            } else if (s.kind == 2) {
                if (pythUpdate.length > 0) {
                    bytes[] memory upd = new bytes[](1); upd[0] = pythUpdate;
                    uint256 fee = IPyth(s.feed).getUpdateFee(upd);
                    IPyth(s.feed).updatePriceFeeds{value: fee}(upd);
                }
                PythStructs.Price memory p = IPyth(s.feed).getPriceNoOlderThan(s.pyt
                uint256 scale = uint256(uint32(-1 * p.expo));
                raw[i] = uint256(uint64(p.price)) * 10**(18 - scale);
            }
        }
        uint256 candidate = _weightedMedian(raw);
        if (a.lastPriceE18 != 0) {

```

```

        uint256 maxDelta = (a.lastPriceE18 * a.clampPerUpdateBps) / 10_000;
        uint256 absDelta = candidate > a.lastPriceE18
            ? candidate - a.lastPriceE18 : a.lastPriceE18 - candidate;
        require(absDelta <= maxDelta, "UPDATE_CLAMP");
    }
    if (a.sodPriceE18 != 0) {
        require(candidate <= a.sodPriceE18 * 10, "SOD_UPPER");
        require(candidate * 10 >= a.sodPriceE18, "SOD_LOWER");
    }
    for (uint256 i = 0; i < raw.length; ++i) {
        uint256 dev = raw[i] > candidate ? raw[i] - candidate : candidate - raw[i];
        if (dev * 10_000 / candidate > a.crossDevBps) { a.status = 1; revert("DE"); }
    }
    a.status = 0;
    a.lastPriceE18 = candidate;
    a.lastUpdate = uint64(block.timestamp);
    return candidate;
}

function _weightedMedian(uint256[] memory v) internal pure returns (uint256) {
    uint256 n = v.length;
    for (uint256 i = 0; i < n; ++i) for (uint256 j = i+1; j < n; ++j)
        if (v[j] < v[i]) (v[i], v[j]) = (v[j], v[i]);
    return v[n/2];
}
}

```

proof_of_reserve_adapter.sol — Chainlink PoR Secure Mint guard:

solidity

```

contract proof_of_reserve_adapter {
    AggregatorV3Interface public immutable porFeed;
    address public immutable backedToken;
    uint8 public immutable feedDecimals;
    uint32 public immutable maxStaleSec;
    constructor(address _por, address _token, uint32 _maxStale) {
        porFeed = AggregatorV3Interface(_por); backedToken = _token;
        feedDecimals = porFeed.decimals(); maxStaleSec = _maxStale;
    }
    function reservesE18() public view returns (uint256) {
        (, int256 ans,, uint256 updatedAt,) = porFeed.latestRoundData();
        require(block.timestamp - updatedAt <= maxStaleSec, "POR_STALE");
        require(ans > 0, "POR_NEG");
        return uint256(ans) * 10**(18 - feedDecimals);
    }
    function supplyAfterOK(uint256 supplyAfterE18) external view returns (bool) {
        return reservesE18() >= supplyAfterE18;
    }
}

```

[peg_registry.sol](#), [asset_library.sol](#), [fee_router.sol](#), [transfer_gate.sol](#), and the per-asset plugins ([xau_plugin.sol](#), [xag_plugin.sol](#), [xpt_plugin.sol](#), [xpd_plugin.sol](#), [btc_anchor_plugin.sol](#), [spcx_plugin.sol](#), [usd_basket_plugin.sol](#)) follow the same pattern. [xau_plugin.sol](#) wraps PAXG at [0x45804880De22913dAFE09f4980848ECE6EcbAf78](#), points its oracle adapter at Chainlink XAU/USD [0x214eD9Da11D2fbc465a6fc601a91E62EbEc1a0D6](#) and the Pyth Metal.XAU/USD feed at Pyth Ethereum contract [0x4305FB66699C3B2702D4d05CF36551390A4c69C6](#), and consults the Paxos-published PoR feed for [supplyAfterOK](#). [GitHub + 2](#)

Algopy parallel implementation

The Algorand parallel recasts the architecture as a [PegFactoryApp](#) ARC4 application using Box storage for per-peg [BasketParams](#) and Inner Transactions to create the per-peg ASA. Because Algorand has no EIP-1167 clones, each peg is itself an ASA with the factory app as manager / reserve / freeze / clawback, and a sibling [PegLogicApp](#) (also created via inner txn) holding the basket composition in a [BoxMap](#). The factory enforces the x402-style deployment fee using GoPlausible metering. Oracle data is pulled via State Proofs (Algorand Chainlink State Proof bridge) or Pyth on Algorand at the appropriate app ID.

python

```

# peg_factory_app.py
# (c) 2026 BANKON – all rights reserved. Apache 2.0. Cypherpunk2048 standard.
# Python >= 3.12, algopy >= 1.0
from typing import TypeAlias
from algopy import (ARC4Contract, BoxMap, Global, Txn, UInt64, arc4, gtxn, itxn, op,

AssetKey: TypeAlias = arc4.StaticArray[arc4.Byte, arc4.Literal[32]]

class BasketComponent(arc4.Struct):
    asset_key: AssetKey
    target_units_e6: arc4.UInt64
    oracle_app_id: arc4.UInt64
    plugin_app_id: arc4.UInt64
    weight_bps: arc4.UInt16

class BasketParams(arc4.Struct):
    name: arc4.String; symbol: arc4.String; decimals: arc4.UInt8
    components: arc4.DynamicArray[BasketComponent]
    unit_of_account: AssetKey
    asset_class: arc4.UInt8
    entry_fee_bps: arc4.UInt16; exit_fee_bps: arc4.UInt16
    streaming_bps_yr: arc4.UInt16
    manifest_cid: arc4.String

class PegRecord(arc4.Struct):
    deployer: arc4.Address
    asa_id: arc4.UInt64; logic_app_id: arc4.UInt64
    basket_hash: AssetKey
    asset_class: arc4.UInt8; deployed_at: arc4.UInt64

class peg_factory_app(ARC4Contract):
    def __init__(self) -> None:
        self.pegs = BoxMap(arc4.UInt64, PegRecord, key_prefix="peg")
        self.basket_params = BoxMap(arc4.UInt64, BasketParams, key_prefix="bp")
        self.registered_plugin = BoxMap(arc4.UInt64, arc4.Bool, key_prefix="plg")
        self.next_peg_id = UInt64(0)
        self.deployment_fee = UInt64(100_000_000) # 100 USDC (6 dp)
        self.usdc_asa = UInt64(31566704) # Algorand mainnet USDC
        self.senatus = arc4.Address(Global.creator_address)

    @arc4.abimethod
    def register_plugin(self, plugin_app_id: arc4.UInt64) -> None:
        assert Txn.sender == self.senatus.native, "ONLY_SENATUS"

```

```

        self.registered_plugin[plugin_app_id] = arc4.Bool(True)

@arc4.abimethod
def create_peg(self, params: BasketParams,
               fee_payment: gtxn.AssetTransferTransaction) -> arc4.UInt64:
    assert fee_payment.xfer_asset.id == self.usdc_asa, "WRONG_ASA"
    assert fee_payment.asset_receiver == Global.current_application_address
    assert fee_payment.asset_amount >= self.deployment_fee, "FEE_LOW"
    n = params.components.length
    assert n.native > 0 and n.native <= 10, "BAD_LEN"
    total_weight = UInt64(0)
    for i in urange(n.native):
        c = params.components[i].copy()
        assert self.registered_plugin[c.plugin_app_id], "PLUGIN_UNREG"
        total_weight += c.weight_bps.native
    assert total_weight == 10000, "WEIGHTS_NOT_100PCT"
    asa = itxn.AssetConfig(
        total=10**18, decimals=params.decimals.native, default_frozen=False,
        unit_name=params.symbol.native, asset_name=params.name.native,
        manager=Global.current_application_address,
        reserve=Global.current_application_address,
        freeze=Global.current_application_address,
        clawback=Global.current_application_address, fee=0
    ).submit()
    logic = itxn.ApplicationCall(
        approval_program=arc4.Bytes(b"<PEG_LOGIC_APPROVAL>"),
        clear_state_program=arc4.Bytes(b"<PEG_LOGIC_CLEAR>"),
        global_num_uint=8, global_num_byte_slice=8, fee=0
    ).submit()
    peg_id = self.next_peg_id + 1
    self.next_peg_id = peg_id
    basket_hash = arc4.StaticArray.from_bytes(op.sha256(params.bytes))
    self.pegs[arc4.UInt64(peg_id)] = PegRecord(
        deployer=arc4.Address(Txn.sender),
        asa_id=arc4.UInt64(asa.created_asset.id),
        logic_app_id=arc4.UInt64(logic.created_app.id),
        basket_hash=basket_hash, asset_class=params.asset_class,
        deployed_at=arc4.UInt64(Global.latest_timestamp))
    self.basket_params[arc4.UInt64(peg_id)] = params.copy()
    return arc4.UInt64(peg_id)

```

A sibling `peg_logic_app.py` implements `mint`, `redeem`, `nav_e6`, and `refresh_all`. The Algorand `oracle_aggregator_app.py` calls a State Proof verifier app for Chainlink data or Pyth

on Algorand, with the same HIP-3 clamps enforced in TEAL via `Global.latest_timestamp - last_update >= min_gap`.

Foundry test suite skeleton

`foundry.toml` sets `solc = "0.8.25"`, `optimizer = true`, `runs = 1_000_000`, with a `[profile.fork]` carrying mainnet, base, and polygon RPC URLs. Layout: `/test/unit/`, `/test/fuzz/`, `/test/invariant/`, `/test/fork/`. Representative scaffolding:

```
solidity

// test/unit/peg_factory.t.sol
contract PegFactoryUnit is Test {
    function test_createPeg_metals_basket() public {
        BasketComponent[] memory cs = new BasketComponent[](4);
        cs[0] = BasketComponent(keccak256("XAU"), 1e18, XAU_ORACLE, XAU_PLUGIN, 4000);
        cs[1] = BasketComponent(keccak256("XAG"), 70e18, XAG_ORACLE, XAG_PLUGIN, 300);
        cs[2] = BasketComponent(keccak256("XPT"), 1e18, XPT_ORACLE, XPT_PLUGIN, 2000);
        cs[3] = BasketComponent(keccak256("XPD"), 1e18, XPD_ORACLE, XPD_PLUGIN, 1000);
        // ... build BasketParams, fund alice with USDC, approve, createPeg ...
    }
}

// test/fork/mainnet_paxg.t.sol - fork against Ethereum mainnet
contract MainnetPAXG is Test {
    function setUp() public { vm.createSelectFork(vm.envString("ETH_RPC_URL")); }
    function test_xau_plugin_reads_chainlink_and_paxg() public {
        xau_plugin plugin = new xau_plugin(
            0x214eD9Da11D2fbe465a6fc601a91E62EbEc1a0D6, // Chainlink XAU/USD
            0x45804880De22913dAFE09f4980848ECE6EcbAf78, // PAXG
            address(0));
        plugin.refresh();
        (uint256 lo, uint256 hi) = plugin.priceE18();
        assertGt(lo, 1_000e18); assertLt(hi, 10_000e18);
    }
}
```

Fuzz tests sweep price deltas from 0 to 5× the previous mark, asserting clamp reverts above 50 bps and SOD-rail reverts above 10× SOD, and that the weighted median is robust to one corrupted source out of three. The invariant test enforces $\text{totalSupply} * \text{NAV} \leq \text{sum}(\text{plugin.erc20.balanceOf(peg)} * \text{price})$ across all registered pegs.

AgenticPlace (agenticplace.pythai.net) discovers deployed pegs by indexing the `PegCreated` event from `peg_registry`. Each peg, on deployment, registers itself with AgenticPlace via `agentRegistry.registerAgent(address(this), manifestCid, assetClass)` routed through the BONAFIDE Tabularium event hub. Subsequent agent metadata (mandate, rebalancing policy, supported jurisdictions, MCP tool schemas for mint / redeem / quote) is pinned to IPFS and the CID lives in both the on-chain `PegRecord` and the off-chain AgenticPlace catalog. mindX (mindx.pythai.net) consumes the same registry as a tool catalog — every peg becomes an MCP server endpoint exposing `nav`, `mint(quote)`, `redeem(quote)`, `basket_composition`, and `jurisdiction_check` tools. BANKON identity is the off-chain KYC oracle whose EIP-712 attestations are verified on-chain by `transfer_gate` calling into BONAFIDE Fides; a Reg-S peg simply registers Fides as its transfer gate and configures the per-jurisdiction allowlist. allchain.html serves as the canonical CAIP-2 chain registry that the factory deployment scripts read to know which template address to use on which chain. Parsec wallet / x402 Algorand routes the deployment fee through GoPlausible for metering, with the receipt logged in the Algorand parallel `peg_factory_app`.

The DELTAVERSE Debt Inheritance integration is the most powerful piece. Each of the 12 EVM contracts (BitcoinAnchorOracle, DebasementIndexV2, InverseDebtToken, GlobalCurrencyBasket, SovereignDebtRegistry, etc.) is wrapped in an `ICollateralPlugin`-conformant adapter and registered in the canonical asset library. A deployer can then compose a `BasketParams` where, for example, 60% is XAU + XAG + XPT + XPD, 30% is the GlobalCurrencyBasket plugin (a synthetic basket of USD / EUR / JPY / CHF / GBP), and 10% is the BitcoinAnchorOracle plugin (itself wrapping the equity-feed layer plus the BTC treasury NAV). The result is a single ERC-20 peg whose name might be "BANKON Sovereign Hedge" with symbol "bnkSH", whose backing is provably 1:1 by Chainlink PoR for the metals and by on-chain transparency for the crypto components, and whose mandate is "preserve purchasing power against sovereign debasement while capturing precious-metal upside."

EasyBasket recipe examples

ebPM — **Precious Metals**: 40% XAU / 30% XAG / 20% XPT / 10% XPD, USD unit-of-account, backed by PAXG, KAG, and Pyth-priced XPT / XPD plugins, with Chainlink XAU/USD and XAG/USD as dominant sources and Pyth metal feeds as cross-reference. Mint via cash (USDC) routed through Uniswap V4 to acquire components; redeem in-kind. Entry/exit 25 bps each, streaming 50 bps/yr for storage/insurance pass-through.

ebSPCX+ — **BTC-anchored pre-IPO equity**: 80% the SPCX reference (via BitcoinAnchorOracle wrapping the Hyperliquid HIP-3 SPCX-USDC mark price and the Paimon SPCX BNB token at `0x872109274218cB50F310E2bFb160D135B502A9d5`) and 20% BTC (Chainlink BTC/USD + WBTC reserves). USD unit-of-account, accredited-investor TransferGate gating Reg-D via BANKON-

Fides attestations. Mint and redeem in-kind only. Streaming 100 bps/yr, performance 1000 bps over a USD high-watermark.

ebFX5 — **Sovereign currency basket:** 30% USD / 30% EUR / 15% JPY / 15% CHF / 10% GBP via Chainlink + Pyth FX feeds. XAU unit-of-account, so the basket measures itself against gold and functions as a debasement-tracker pair-trade against **ebPM**.

ebENG — **Commodity basket:** 50% WTI crude (Pyth USOILSPOT / UKOILSPOT) + 30% natural gas + 20% gold as inflation hedge. USD unit-of-account.

ebRWA — **Mixed RWA:** 25% ebPM + 25% ebFX5 + 25% GlobalCurrencyBasket plugin + 25% ebSPCX+. The registry explicitly allows other EasyBasket pegs as components provided each sub-peg is itself in the asset library, giving a Reserve-style two-level basket hierarchy.

Deployment scripts

Scripts at `script/deploy_eth.s.sol`, `script/deploy_base.s.sol`, `script/deploy_polygon.s.sol`, `script/deploy_arc_testnet.s.sol`, and `script/deploy_algorand.py` each construct a `HelperConfig` reading the chain-specific feed addresses (XAU/USD `0x214eD9Da11D2fbe465a6fc601a91E62EbEc1a0D6` on Ethereum, XAG/USD `0x379589227b15F1a12195D3f2d90bBc9F31f95235` on Ethereum, Pyth contract `0x4305FB66699C3B2702D4d05CF36551390A4c69C6` on Ethereum, with equivalents pulled from docs.chain.link/data-feeds/price-feeds/addresses and docs.pyth.network/price-feeds/contract-addresses/evm for Base, Polygon, and Arbitrum). On Arc Testnet 5042002 the scripts deploy the same bytecode with mock Chainlink and mock Pyth contracts; Algorand uses AlgoKit + algopy via `algokit project deploy --network mainnet`.

Recommendations

Ship in three staged releases. **T+30:** Ethereum + Base deployment of `peg_factory`, `peg_token_template`, `peg_registry`, `asset_library`, `oracle_aggregator`, the four metal plugins (PAXG-XAU, KAG-XAG, Pyth XPT, Pyth XPD), and the `ebPM` reference peg. Audit budget is a minimum two-firm audit (recommend Trail of Bits — which has audited Paxos USDP and PYUSD per the paxosglobal/usdp-contracts GitHub repository — paired with OpenZeppelin, which audited Set Protocol V2 core contracts per docs.indexcoop.com/index-coop-community-handbook/protocol/security-and-audits) covering factory, template, composer, aggregator, and PoR adapter. The benchmark gating production: every metal plugin returns `SOUND` on a Foundry mainnet fork for 30 consecutive blocks; the aggregator rejects all out-of-clamp updates in a 10,000-iteration fuzz; the invariant `totalSupply * NAV ≤ reserves` holds for 100,000 random mint/redeem sequences.

T+60: cash-mint via Uniswap V4 hooks, `transfer_gate` integrated with BONAFIDE Fides for KYC, the BTC-anchored equity peg `ebSPCX+`, and the sovereign currency basket `ebFX5`. Gating

threshold: a successful end-to-end mint on Arc Testnet 5042002 by a wallet holding a valid BANKON-signed Reg-D accreditation attestation, plus successful redemption.

T+90: Algorand parallel `peg_factory_app` and `peg_logic_app`, GoPlausible x402 metering, AgenticPlace agent-registry hooks, mindX MCP tool generation, and the DELTAVERSE Debt Inheritance plugin family. Gating threshold: one live cross-chain peg whose Algorand ASA total supply matches its EVM ERC-20 total supply within $\pm 0.01\%$ for 7 consecutive days, verified by a Chainlink CCIP-style reserve check.

Adopt Reserve's three-state collateral status (`SOUND`) / (`IFFY`) / (`DISABLED`) verbatim — proven minimum surface for handling oracle anomalies and asset defaults without governance intervention. Adopt the HIP-3 clamp constants verbatim (± 50 bps per update, $\pm 1\%$ per mark step, $\pm 10\times$ per day, 2.5 s minimum gap) until live data shows they need tightening; loosening them is dangerous, tightening is cheap. Cap component count at 10 per Index Coop's empirical lesson. Make the factory UUPS-upgradeable behind a Senatus-controlled timelock, but keep each deployed peg immutable as a hard guarantee to holders. Do not custody collateral in the factory — every collateral plugin holds its own collateral ERC-20 directly, mirroring Set V2's design where *"all the components of a SetToken are held in the SetToken contract itself."* Wire Chainlink PoR Secure Mint into every physically-backed plugin's `proofOfReserveOK()` from day one — cost is one extra `staticcall` per mint, value is eliminating the entire infinite-mint attack class. `Tokensets`

Caveats

The most significant epistemic caveat is the limitation of Chainlink Proof of Reserve when the issuer is the sole reporter, documented by Will Canny in CoinDesk on July 5, 2023 ("Chainlink 'Proof of Reserve' Proves Little Beyond Data Going In, Coming Out"): *"Of the 16 third-party node operators that report on PAXG's gold reserves, every single one of them gets its data from the same place: Paxos itself."* PoR is therefore a transparency upgrade and an infinite-mint shield, not a true third-party audit. Any peg backed by PAXG, XAUT, or other custodian-attested assets inherits the issuer's counterparty risk in full, and the on-chain manifest CID must disclose this explicitly. `CoinDesk`

The exact 0x address of Reserve Protocol's Yield-DTF DeployerRegistry on Ethereum mainnet was not directly resolvable from the GitHub README link in the time available — the developer docs link to it as "here" but the precise bytes should be confirmed by clicking through before any Foundry script hard-codes it. The Index DTF FolioDeployer addresses given above (`0x4d201a6e5bf975e2cee9e5cbdfc803c0ff122073`) on Ethereum, (`0x3451fD177E9a8bB4Eb8271E627A804BD22A816F9`) on Base, both v5.0.0) are taken from docs.reserve.org/core-components/index-dtfs/smart-contracts.md and were directly verified; treat the Yield-DTF DeployerRegistry address as "to be confirmed at deployment time."

`Reserve +2`

The Pyth feed IDs for XAU/USD (truncated form `0x765d...4bb2`) per insights.pyth.network/price-feeds/Metal.XAU%2FUSD) and XAG/USD were only partially fetchable from Pyth Insights in this session due to client-side rendering. The full 32-byte values should be confirmed via the Pyth Hermes API at https://hermes.pyth.network/v2/price_feeds?asset_type=metal before any Solidity constant is hard-coded. [Pyth](#)

The HIP-3 clamp constants are correct as of the live trade[XYZ] equity perps deployment quoted from docs.trade.xyz, but Hyperliquid documents that for HIP-3 perps specifically a "more responsive premium formula" may be used by deployers to express larger ranges; cypherpunk2048 PegFactory adopts the conservative trade[XYZ] values by default and exposes them as Senatus-configurable parameters per asset rather than hard-coding them.

[Hyperliquid Docs](#)

Several long-tail competitors were intentionally not adopted as primary references: TokenSets / Set V1 (superseded by V2), Balancer weighted pools (price-from-pool design unsuitable for RWA), Synthetix (synthetic, no physical backing), mStable (deprecated). dHEDGE and Enzyme are viable for the off-chain manager pattern but out of scope for the on-chain composable peg use case Gregory has specified.

Finally, the legal posture of any peg whose basket includes pre-IPO equity (the SPCX path) must be set by counsel before mainnet shipment; the technical TransferGate / Fides integration is necessary but not sufficient for Reg-D / Reg-S compliance, and the manifest CID must contain the issuer-side disclosure document that AgenticPlace surfaces as the canonical "investor information sheet" before any subscription.